

Optimization of Pearson and Blur

DV1674 | Assignment 2



Alexander Järnström
aljr21@student.bth.se

Jesper Lennehag
jele22@student.bth.se

CONTENTS

I. Introduction	1
II. Method	1
III. Pearson	1
IV. Blur	2
V. Results	2

I. INTRODUCTION

II. METHOD

III. PEARSON

A simple bash script was written, `tests.sh`, with the purpose of making the testing and verification simpler. It has two modes: verify the output of the binary to previously established results and run the performance tests, which was used to extract the data. It ran all the tests on all the provided data files containing the various sizes of vectors, and later all threads. The parallel version of the binary is verified by the provided verification script, named `verify.sh`, making the verification part of `tests.sh` redundant and removed, only leaving the different performance tests.

The initial step taken was to enable compiler optimizations with the flags `-O2` and `-O3` where the one with best time was kept, `-O3` proved to be 527ms quicker than `-O2` and brought the total time down with 71.8% from the original. All the other stages are using the `-O3` flag when compiling.

Table 1: Changes between the Base and the O3 step.

Time	N/A
Cache Miss	N/A
CPU usage	N/A
Memory Usage	N/A
Instructions	N/A

Next was the implementation of a cache, keeping all the results from the mean and magnitude methods in the Vector class. This was achieved by creating two arrays `mean_cache` and `magnitude_cache` which were initialized with the maximum value a double can take. It would check the cache, if the value equals the maximum a value for mean and magnitude would be calculated and then stored at the provided index, otherwise the value would be collected and used. This subtle change pulled the time down with another 32.67% from the previous step and 74.4% from the original.

The third step took inspiration from locality, moving the calculations into the vector. VTune showed massive allocation amounts, for size 1024 the total reached 30.2 GB, where the biggest corporate is the constant copying and construction of new vectors when performing all the calculations. This step took the cache and *moved* it into the vector, or with other words replaced it with the vector itself. By realizing the original vector wasn't needed, all the calculations were changed to be

performed directly on the original. This replaced the original values by the calculated ones storing them in the vector, saving them for the dot method. This step lowered the allocation amount down to 103.9 MB for 1024, meaning the requests for more memory to the OS fell giving a positive difference of 40.31% from the caching step and 88.64% difference from the original.

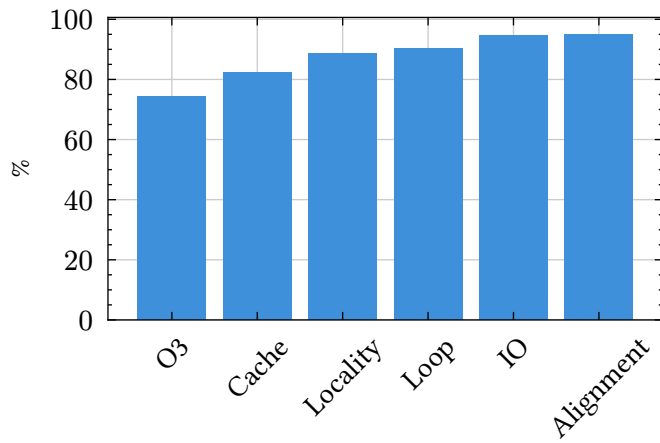
We continued to loop unfolding, allowing the loops in the vector to do four calculations per iteration and eliminating dependencies by doing said calculations to temporary variables and then calculating the final result from them minimizing the pipeline stalls. This simple change lowered the time by 17.62% from the previous step and 90.2% from the original, leaving it at 704ms for size 1024.

The fifth step focused on IO the functions taking the most time now were `dot`, `read`, and `write`. `write` is heavily IO bound due to reading and writing file content to the disk, `write` was optimized by removing `std::endl` from the stream and replacing it with `\n` this allowed the stream to continue without clearing the buffer every iteration through the results. This gave us another 45.16% off the previous and a total of 94.64% from the original, giving us 404ms for 1024.

The final step taken is vector alignment, by using `alignas(double)`. Aligning the vector class to the primary value in the data array, gave us a slight performance increase. This lowered the L1 cache misses, by 0.21% and LL cache misses by and gave us a speed increase of 6.05% from the previous step and 94.93% from the original. Leaving us at the final time of 376ms for size 1024 compared to 20,490ms for the original with the same size.

All the changes are illustrated in Graph 1 showing all the time differences through out the steps on different vector sizes. Due to the size 1024 over shadowing the other values, Graph 2 might be better to look at. The second graph shows the removed time in percent compared to the original while Graph 3 shows the removed time in percent from the previous step, or with other words, the impact.

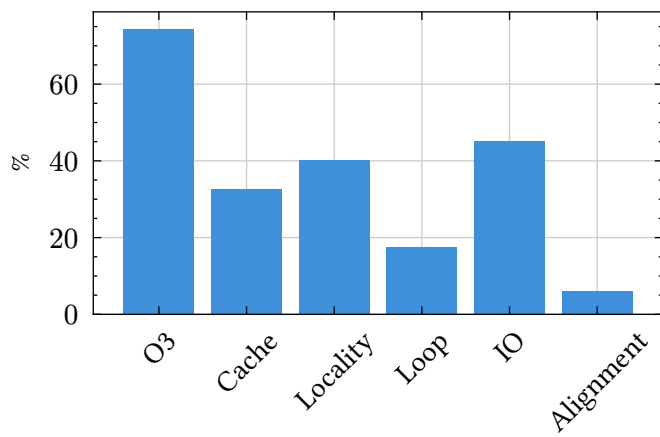
Several steps to optimize the program were taken, such as caching, loop unfolding, and simple optimisation flags. We will only show the ones actually providing a positive time difference, meaning manual vectorisation using libraries such as `immintrin.h` was discarded due to worsening the time with 50ms.



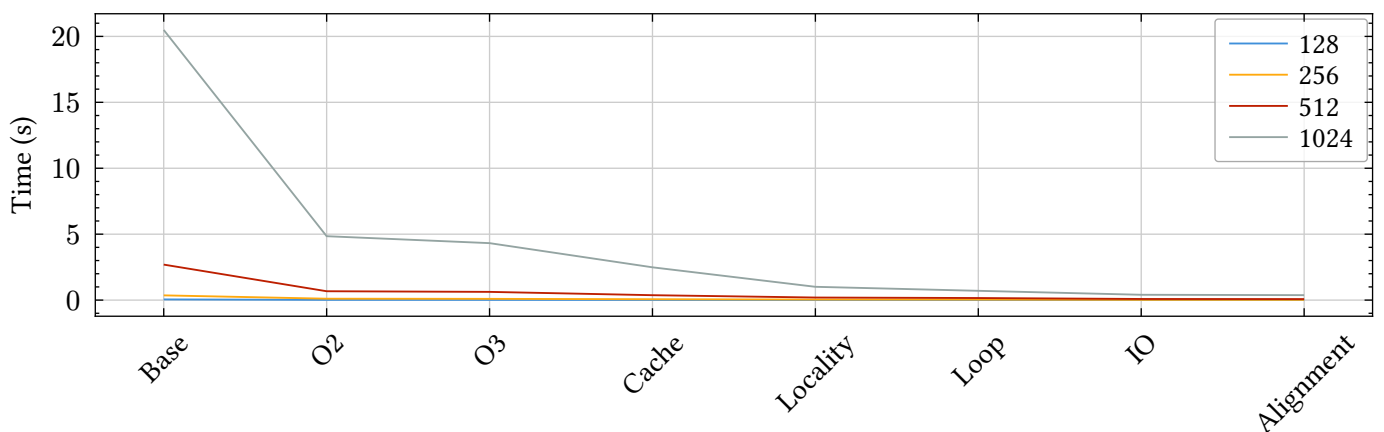
Graph 2: Illustration of the total removed time in percent from the base.

IV. BLUR

V. RESULTS



Graph 3: Illustration of the total removed time in percent from the previous step.



Graph 1: Illustration of all the different sizes through all the take steps.